

Redis SQL Trino

Version 0.4.2

Table of Contents

Introduction	1
Requirements and compatibility	2
Quick start with a released plugin	3
Installation and upgrades	5
Configuration	6
Querying existing data	10
SQL support	12
Execution and correctness	16
Performance and concurrency	20
Limitations	23
Clients	24
Troubleshooting	25
Building from source	27
Single-node Linux walkthrough	28

Introduction

Redis SQL Trino exposes Redis Query Engine indexes as tables in [Trino](#). Use SQL to read Redis hashes and JSON documents, join them with other Trino catalogs, and connect JDBC clients such as Tableau or Apache Superset. The connector can also create and modify hash-backed tables.

Redis evaluates supported filters, aggregations and limits close to the data. Trino evaluates the remaining SQL, including joins, window functions and expressions the connector cannot translate exactly. An index is required: this connector discovers tables with `FT._LIST` and reads them with `FT.AGGREGATE` and cursors, rather than scanning the Redis keyspace.

The connector name and configuration prefix remain `redisearch`, even though Redisearch is now called the Redis Query Engine. This guide describes the current connector source; use a release with the matching Trino version when installing it.

- [Requirements and compatibility](#)
- [Quick start with a released plugin](#)
- [Installation and upgrades](#)
- [Configuration reference](#)
- [Querying existing HASH and JSON data](#)
- [SQL support and data types](#)
- [Execution, pushdown and numeric correctness](#)
- [Performance and parallel scans](#)
- [CLI, JDBC and visualization clients](#)
- [Troubleshooting](#)
- [Building from source](#)
- [Single-node Linux walkthrough](#)

Requirements and compatibility

Component	Requirement
Trino	483. The connector uses Trino's plugin interface; do not assume a plugin built for another Trino release is compatible.
Java	64-bit Java 25, at least 25.0.1, for Trino 483. Use Eclipse Temurin for building this project; its Maven rules accept Temurin or Oracle. The stock Trino container includes its runtime.
Redis	A database with the Query Engine and working <code>FT._LIST</code> , <code>FT.INFO</code> , <code>FT.SEARCH</code> , <code>FT.AGGREGATE</code> and <code>FT.CURSOR</code> commands. HASH and JSON indexes are readable; creating and updating rows requires HASH indexes.
Network	The coordinator and every Trino worker must reach the Redis endpoint. With the OSS Cluster API enabled, they must also reach the addresses advertised by the cluster.

Redis Software and Redis Cloud RAM databases are the default development and benchmark targets. Redis 8 includes the Query Engine; the optional Open Source example uses `redis:8.4`. Redis Stack 7.x also includes Search and JSON, but some aggregation expressions need newer Query Engine capabilities. The connector probes support for `case()` before using the corresponding aggregation path; see [aggregation correctness](#).

Flex is not supported on the targets recorded in the project's testing matrix. Redis Cloud Essentials Flex has no Query Engine, and the documented Cloud Pro Search-on-Flex beta lacks `FT.AGGREGATE` and NUMERIC fields, which this connector needs. The locally tested Redis Software Flex build also rejected required read and DDL operations. See [the tested deployment matrix](#) for pinned versions, checks and limitations, and [Redis's current Flex documentation](#) for product availability. A successful TCP connection or an advertised command alone does not establish connector compatibility.

Quick start with a released plugin

This example runs Trino in Docker against an existing Redis Software or Redis Cloud RAM database with the Query Engine. It does not require a local JDK or Maven build. Use a disposable database for the write example below.

1. Download the release and unpack every plugin JAR into one directory:

```
mkdir -p redis-trino-demo/plugin redis-trino-demo/catalog
cd redis-trino-demo
curl -fL https://github.com/redis-field-engineering/redis-sql-trino/releases/download/v0.4.2/redis-sql-trino-0.4.2.zip -o connector.zip
unzip -j connector.zip -d plugin
```

These release URLs assume the guide was published with a release version. When building a snapshot, use [the source build](#) instead of a snapshot release URL.

2. Create `catalog/redisearch.properties`, replacing the endpoint and credentials:

```
connector.name=redisearch
redisearch.uri=redis://your-redis-endpoint:port
redisearch.username=default
redisearch.password=replace-with-your-password
redisearch.cluster=false
```

Use `redis://` for a database without TLS. For a Redis Software database exposed on host port 12000, use `redis://host.docker.internal:12000`. A container's `localhost` refers to that container, not to the host or another container. Protect this file from other users and source control; see [Authentication and TLS](#).

3. Start Trino with the plugin and catalog mounted:

```
docker run -d --name redis-trino-demo \
  --add-host=host.docker.internal:host-gateway \
  -p 127.0.0.1:8080:8080 \
  -v "$PWD/plugin:/usr/lib/trino/plugin/redisearch:ro" \
  -v "$PWD/catalog:/etc/trino/catalog:ro" \
  trinodb/trino:483
```

The HTTP endpoint is bound to the host's loopback interface for this local demo. Wait for `docker logs redis-trino-demo` to report that the server started, then open its bundled CLI:

```
docker exec -it redis-trino-demo trino --catalog redisearch --schema default
```

4. Check discovery and try a small table:

```
SHOW TABLES;
CREATE TABLE guide_drinks (id VARCHAR, name VARCHAR, abv DOUBLE);
INSERT INTO guide_drinks VALUES ('1', 'Test Ale', 5.5), ('2', 'Light Ale', 3.2);
SELECT __key, name, abv FROM guide_drinks WHERE abv > 4;
SELECT count(*), avg(abv) FROM guide_drinks;
UPDATE guide_drinks SET abv = 5.7 WHERE id = '1';
DELETE FROM guide_drinks WHERE id = '2';
```

The filtered query returns **Test Ale**; the first aggregate returns a count of **2** and an average of **4.35**. Each INSERT creates a new Redis hash with a generated key; **id** is an ordinary column, not a primary key. Do not rerun INSERT expecting it to replace rows.

5. Clean up the example table only after confirming it contains no data you want to keep:

```
DROP TABLE guide_drinks;
```

DROP TABLE deletes the index and its documents. Exit the CLI and remove the demo container with `docker rm -f redis-trino-demo`. The Redis database and the local plugin/catalog files remain.

For the repository's Compose-based development environment and optional Open Source beers demo, see [the README quick start](#).

Installation and upgrades

Install Trino 483 using [Trino's deployment instructions](#), or follow the [single-node Linux walkthrough](#). On every coordinator and worker, unpack the connector ZIP into `<trino>/plugin/redisearch` with all its JARs at that directory's top level. A container uses `/usr/lib/trino/plugin/redisearch`. Installing only the connector's own JAR leaves its dependencies missing.

Create `etc/catalog/redisearch.properties` on each node (or `/etc/trino/catalog/redisearch.properties` in the stock container). The filename defines the catalog name: `analytics.properties` exposes `analytics.default.<index>`, while `connector.name=redisearch` stays the same. All nodes need matching plugin versions, catalog settings and readable certificate files. Restart the affected Trino nodes after installing the plugin or changing their static catalog configuration.

For an upgrade, stop the node, replace the plugin directory's contents with the new distribution, and restart it. Do not overlay a new ZIP onto old dependency JARs. Back up catalog files separately and verify `SHOW TABLES`, `DESCRIBE` and representative reads before resuming workloads. An upgrade does not rewrite application keys or indexes.

Configuration

Catalog properties configure the connector at startup. The following is a minimal non-TLS local connection:

etc/catalog/redisearch.properties

```
connector.name=redisearch
redisearch.uri=redis://localhost:6379
```

Table 1. Connector properties

Property name	Description	Default
<code>redisearch.default-schema-name</code>	Name of the single SQL schema containing the Redis indexes. This does not select a Redis logical database or key prefix.	<code>default</code>
<code>redisearch.case-insensitive-names</code>	Match index names case insensitively.	<code>false</code>
<code>redisearch.cursor-count</code>	Number of rows requested per aggregation cursor batch. <code>0</code> uses the server's batch default; it does not cap the total query result.	<code>1000</code>
<code>redisearch.query-timeout-ms</code>	Redis aggregation timeout override in milliseconds. <code>0</code> leaves the server default unchanged. This does not replace Trino's query timeout; see timeouts and result completeness .	<code>0</code>
<code>redisearch.narrow-scan-cursor-count</code>	Optional batch size for unbounded document scans of at most two requested columns. Maximum 10000; <code>0</code> uses <code>cursor-count</code> . Applies to the initial request and later cursor reads. Larger batches increase response/hydration memory; only one cursor read is prefetched.	<code>0</code>
<code>redisearch.aggregation-group-limit</code>	Conservative GROUP BY budget; must not exceed the server MAX_AGGREGATE_GROUPS. Unknown or over-budget document counts retain Trino aggregation. Global aggregates remain eligible.	<code>1000000</code>
<code>redisearch.scan-connections</code>	Number of Redis connections each Trino node's scans take turns on, so that one scan's batches don't hold up other queries' commands.	<code>8</code>
<code>redisearch.scan-splits</code>	<code>0</code> selects splits automatically from size, estimated CPU capacity and partition balance. <code>1</code> forces a single split; <code>2</code> to <code>64</code> explicitly bounds parallel scans with configured cut points. The connection pool also caps the count. Global operations remain single-split.	<code>0</code>

Property name	Description	Default
<code>redisearch.scan-partition-field</code>	Optional indexed NUMERIC field alias used to partition scans. Automatic mode can discover a HASH field when unset. Missing, NOINDEX, unsupported or unsafe field names fall back to one split.	Unset
<code>redisearch.scan-partition-boundaries</code>	Optional comma-separated strictly increasing finite numeric cut points; automatic mode can discover them for HASH indexes. Prefer a stable, evenly distributed field. Missing values stay in the first partition; JSON arrays spanning cut points appear once.	Empty
<code>redisearch.aggregation-pushdown.enabled</code>	Compute supported aggregations in Redis. Set to <code>false</code> to compute them in Trino from cursor batches when large aggregations interfere with latency-sensitive queries. This transfers more rows and can make aggregations substantially slower. Filters still push down.	<code>true</code>
<code>redisearch.table-cache-refresh</code>	Seconds after which the connector describes a cached index again (<code>FT.INFO</code> , its saved column types and a few documents), in the background: queries keep planning with the cached schema meanwhile. An index no query reads for ten times as long is dropped from the cache, and the next query waits for it. Fields added and indexes dropped outside Trino can take this long to show. <code>0</code> disables the cache.	<code>60</code>
<code>redisearch.dynamic-filtering.enabled</code>	Add the join keys that the other side of a join collects (its dynamic filter) to the query that scans an index.	<code>true</code>
<code>redisearch.dynamic-filtering.wait-timeout</code>	How long a scan waits for the other side of a join to collect its dynamic filter before it starts without it.	<code>20s</code>

Table 2. Redis connection properties

Property name	Description	Default
<code>redisearch.uri</code>	A Redis connection string. Redis URI syntax .	
<code>redisearch.username</code>	Redis connection username.	
<code>redisearch.password</code>	Redis connection password.	
<code>redisearch.cluster</code>	Connect to a Redis Cluster.	<code>false</code>
<code>redisearch.resp2</code>	Force Redis protocol version to RESP2. Trino then computes the aggregations whose results or <code>GROUP BY</code> keys are DOUBLE or REAL: see Pushdown .	<code>false</code>

The Redisearch connector provides additional security options to support Redis servers with TLS mode.

Table 3. TLS properties

Property name	Description	Default
<code>redisearch.insecure</code>	Allow insecure connections (e.g. invalid certificates) when using SSL.	<code>false</code>
<code>redisearch.cacert-path</code>	X.509 CA certificate file to verify with.	
<code>redisearch.key-path</code>	PKCS#8 private key file to authenticate with (PEM format).	
<code>redisearch.key-password</code>	Password of the private key file, or null if it's not password-protected.	
<code>redisearch.cert-path</code>	X.509 certificate chain file to authenticate with (PEM format).	

Authentication, TLS and cluster routing

Use `rediss://` to enable TLS; certificate properties alone do not change a `redis://` connection into a TLS connection. Set the ACL username and password with `redisearch.username` and `redisearch.password`, or use the authentication portion of a Redis URI. When a separate password is set, it overrides URI credentials; a separate username is applied with that password. Avoid putting passwords in shell history or committed configuration files. For deployments using Trino's secrets support, see [Trino secrets management](#).

A TLS connection with separate credentials can look like this:

```
connector.name=redisearch
redisearch.uri=rediss://redis.example.net:12000
redisearch.username=analytics
redisearch.password=${ENV:REDIS_ANALYTICS_PASSWORD}
redisearch.cacert-path=/etc/trino/certs/redis-ca.pem
```

The environment variable must exist in every Trino node's process. Omit `cacert-path` when the default trust configuration already trusts the server certificate. For mutual TLS, also configure `redisearch.cert-path` and `redisearch.key-path`, and `redisearch.key-password` when the private key is encrypted. Certificate paths refer to files inside the Trino runtime; mount them into containers on every node. `redisearch.insecure=true` disables peer verification and should be limited to controlled diagnostics.

Set `redisearch.cluster=true` only when the database exposes the OSS Cluster API. A sharded Redis Software or Cloud database behind a single database endpoint can still use `cluster=false`; sharding alone does not imply cluster routing. Check the database's connection mode and advertised node addresses rather than deriving this setting from its shard count.

An ACL must allow the index commands and access to the covered document keys. Discovery reads `FT._LIST`, `FT.INFO`, `FT.SEARCH` and `GET __trino:columns:<index>`; scans use `FT.AGGREGATE`, `FT.CURSOR` and, when needed, exact hash reads. Writes additionally need the hash/delete commands for their statements, and DDL needs `FT.CREATE`, `FT.ALTER` or `FT.DROPINDEX` plus access to the type metadata keys. Scope permissions to the indexes, document prefixes and operations the deployment uses,

and test with the actual service account. Allowing **PING** alone is insufficient.

Removed properties

Do not carry **redisearch.default-limit** or **redisearch.table-cache-expiration** into a new catalog. Both are declared defunct and rejected at startup. The former silently capped scans; use a SQL **LIMIT** for an intentional result limit. Use **redisearch.table-cache-refresh** for metadata refresh.

Querying existing data

Use Redis **FT.CREATE** to expose hashes or JSON documents your applications already write. **CREATE TABLE** creates a new HASH index with **SKIPINITIALSCAN**, so it is not the way to index an existing keyspace. Only indexed documents become SQL rows; unrelated Redis strings, lists and unindexed hashes do not appear as tables.

HASH example

Run these commands with a Redis CLI connected to your database, using an unused key prefix and index name:

```
HSET guide:beer:1 id 1 name "Test Ale" abv 5.5
HSET guide:beer:2 id 2 name "Light Ale" abv 3.2
FT.CREATE guide_beers ON HASH PREFIX 1 guide:beer: SCHEMA id TAG name TEXT abv NUMERIC
FT.INFO guide_beers
```

Wait until **FT.INFO** reports **indexing** as **0**, then query from the Trino CLI:

```
DESCRIBE redisearch.default.guide_beers;
SELECT __key, name, abv
FROM redisearch.default.guide_beers
WHERE abv > 4;
```

abv reads as **DOUBLE**; **id** and **name** read as **VARCHAR**. **NUMERIC** filters can run in Redis. A **TEXT** field's SQL equality is exact string equality, not a request for arbitrary full-text matching; see [filter semantics](#).

JSON example

Create a JSON document and an index with explicit field aliases:

```
JSON.SET guide:product:1 $ '{"sku":"A1","name":"Travel Mug","price":12.5}'
FT.CREATE guide_products ON JSON PREFIX 1 guide:product: SCHEMA $.sku AS sku TAG
$.name AS name TEXT $.price AS price NUMERIC
FT.INFO guide_products
```

After background indexing finishes:

```
SELECT __key, sku, price, "$"
FROM redisearch.default.guide_products
WHERE price < 20;
```

The SQL columns use the aliases **sku**, **name** and **price**, rather than the JSON paths. The **"\$"** column

contains the full document as JSON text. Indexing a JSON array does not turn its elements into separate SQL rows; a row represents a document. Use Trino JSON functions to interpret document text when necessary. The connector does not insert or update JSON documents; **DELETE** and the delete branch of **MERGE** can remove them.

Schema discovery and application changes

An index's indexed fields define its primary set of SQL columns. The connector also samples the first 10 documents for unindexed fields; those columns are VARCHAR and are not a complete union of every document's fields. Use explicit indexed fields when a stable SQL schema matters.

Metadata refresh is separate from row freshness. **redsearch.table-cache-refresh** refreshes index descriptions and saved types, not a cached copy of query results. Each query reads Redis; changes to existing fields do not require a schema version or a metadata refresh. A newly added field can remain absent from the cached schema until the background refresh completes.

To remove only an example index while retaining its data, use **FT.DROPINDEX guide_beers** in Redis without **DD**. SQL **DROP TABLE** uses **DD** and deletes the documents too.

SQL support

Tables and columns

Each Redis Query Engine index is a table in the `default` schema (see `redisearch.default-schema-name`). A table has these columns:

- One per indexed field, named by its attribute (the `AS` name for JSON paths).
- One per field that isn't indexed but appears in the first 10 documents returned by `FT.SEARCH <index> *`.
- For JSON indexes, a `$` column holding the whole document as JSON text.
- A hidden `__key` column holding the document's Redis key. Hidden columns are left out of `SELECT *`, so name it: `SELECT __key, * FROM beers`.

Table 4. Type mapping

Redis field type	Trino type
NUMERIC	DOUBLE
TAG, TEXT, GEO, GEOSHAPE, VECTOR	VARCHAR
Not indexed	VARCHAR

Columns created with `CREATE TABLE` or `ALTER TABLE ... ADD COLUMN` read back as the types they were created with instead; see [column types](#).

After `FT.CREATE` over existing keys, or `ALTER TABLE ... ADD COLUMN` (which runs `FT.ALTER`), Redis indexes the documents in the background. Until it finishes, queries on that index fail with `RESEARCH_INDEX_NOT_READY` (Index `beers` is still being built (42% indexed) ...) instead of returning incomplete results. Retry once `FT.INFO` reports `indexing` as `0`. `CREATE TABLE` creates its index with `SKIPINITIALSCAN`, so a new table can be queried right away.

Statements

The catalog exposes one configured schema. `redisearch.default-schema-name` changes its name; it does not create or select a Redis database or key prefix. Use fully qualified `catalog.schema.table` names when a client has no default catalog/schema.

Table 5. Supported statements

Statement	Notes
<code>SELECT</code>	All of Trino's query syntax, including joins (also across catalogs), subqueries, <code>WITH</code> , <code>UNION</code> , <code>DISTINCT</code> , <code>GROUP BY</code> , <code>ORDER BY</code> and window functions. Pushdown describes which parts run in Redis.
<code>SHOW SCHEMAS</code> , <code>SHOW TABLES</code> , <code>SHOW COLUMNS</code> , <code>DESCRIBE</code> , <code>SHOW CREATE TABLE</code> , <code>information_schema</code>	<code>SHOW TABLES</code> lists the indexes returned by <code>FT._LIST</code> .

Statement	Notes
EXPLAIN	The table scan shows the filters, limit and aggregations sent to Redis.
CREATE TABLE	Creates a hash index named after the table, with key prefix <code><table>:</code> . See column types . <code>IF NOT EXISTS</code> is supported.
CREATE TABLE AS SELECT	Columns get the same field types as in <code>CREATE TABLE</code> . A query with a column of another type fails before the table is created.
INSERT, INSERT ... SELECT	Hash indexes only: the connector writes hashes, which an index on JSON documents never sees, so it fails on a JSON index. Each row is a new hash with key <code><prefix><ULID></code> , where <code><prefix></code> is the index's first key prefix.
UPDATE	Hash indexes only. Setting a column to <code>NULL</code> removes the field from the hash.
DELETE	Deletes the matching keys, on hash and JSON indexes.
MERGE	Hash indexes only, except that <code>WHEN MATCHED THEN DELETE</code> also works on JSON indexes.
ALTER TABLE ... ADD COLUMN	Adds a field to the index with <code>FT.ALTER</code> . Columns are always added at the end. Queries on the table fail until Redis finishes reindexing its documents.
DROP TABLE	Drops the index and deletes its documents (<code>FT.DROPINDEX ... DD</code>).

These statements fail with a `This connector does not support ...` error:

- `CREATE SCHEMA, DROP SCHEMA, ALTER SCHEMA`
- `CREATE OR REPLACE TABLE`
- `ALTER TABLE ... RENAME TO, DROP COLUMN, RENAME COLUMN, ALTER COLUMN ... SET DATA TYPE`
- `TRUNCATE`
- `COMMENT ON`
- `CREATE VIEW, CREATE MATERIALIZED VIEW`
- Queries on table versions (`FOR VERSION AS OF, FOR TIMESTAMP AS OF`)

Writes also fail when fault-tolerant execution is enabled (`retry-policy` other than `NONE`).

Writing rows and changing tables

Use a dedicated key prefix for tables whose lifecycle Trino manages. A new table named `guide_orders` uses the index `guide_orders`, document prefix `guide_orders:` and type metadata key `__trino:columns:guide_orders`.

```
CREATE TABLE guide_orders (
  order_id BIGINT,
```

```
customer VARCHAR,
amount DOUBLE,
ordered_at TIMESTAMP(3)
);
INSERT INTO guide_orders
VALUES (1, 'Ada', 12.5, TIMESTAMP '2026-10-09 12:00:00');
UPDATE guide_orders SET amount = 14.0 WHERE order_id = 1;
SELECT __key, * FROM guide_orders WHERE order_id = 1;
ALTER TABLE guide_orders ADD COLUMN note VARCHAR;
```

ALTER TABLE ... ADD COLUMN changes the index and saved type metadata, but does not populate the new field in existing hashes. Missing values read as SQL NULL. Adding an indexed field can trigger background reindexing; wait for it to finish before reading the table.

INSERT sends one pipelined **HSET** per row, using a generated ULID key under the index's first prefix. A prefix without a trailing **:** has one appended; if no usable prefix is found, the writer uses **<table>:**. An INSERT into an externally created index therefore requires a compatible key prefix for its rows to be indexed. NULL values and omitted columns are not written. SQL columns do not declare Redis keys or enforce primary-key uniqueness. Two INSERTs with the same **order_id** create two hashes. UPDATE and MERGE match SQL predicates; they do not turn an ordinary ID column into a uniqueness constraint.

Writes are not transactional across rows. A failed INSERT, UPDATE, DELETE or MERGE can leave completed Redis changes in place; retrying a write may add duplicates or repeat changes. Inspect the affected rows and application keys before retrying. Do not assume Trino rollback reverses all document writes.

CREATE TABLE AS SELECT creates a HASH index and writes the selected rows using the same rules:

```
CREATE TABLE guide_orders_copy AS
SELECT order_id, customer, amount, ordered_at FROM guide_orders;
```

Use **DROP TABLE guide_orders_copy** and **DROP TABLE guide_orders** only when you intend to delete their data. For externally owned indexes, removing an index without its documents must be done in Redis with **FT.DROPINDEX** without **DD**.

Stored column types

Table 6. Column types in CREATE TABLE

Trino type	Index field	Stored in the hash as
BIGINT, INTEGER, SMALLINT, TINYINT, DOUBLE, REAL, DECIMAL	NUMERIC	The number, such as 42, 4.5 or 1.25
TIMESTAMP(3), TIMESTAMP(3) WITH TIME ZONE	NUMERIC	Epoch milliseconds
VARCHAR, CHAR, UUID	TAG	The string, with CHAR padded with spaces

Trino type	Index field	Stored in the hash as
BOOLEAN, DATE	TAG	true or false, and ISO dates such as 2024-01-02

Other types, such as ARRAY, MAP, ROW, VARBINARY and TIME, fail with **Unsupported column type**.

Redis keeps only each field's index type, which columns of several Trino types share. So **CREATE TABLE**, **CREATE TABLE AS SELECT** and **ALTER TABLE ... ADD COLUMN** also save the columns' Trino types, as JSON in the string key `__trino:columns:<table>`, such as `{"regionkey":"bigint","name":"varchar(25)"}`, and **DROP TABLE** deletes the key. Columns read back as those types in **DESCRIBE**, **SHOW CREATE TABLE**, `information_schema` and queries, and **INSERT** takes values of those types. **CREATE TABLE AS SELECT** keeps bounded types, so **SELECT 'a' AS id** makes a `varchar(1)` column.

The types in the key don't apply to:

- Indexes created outside Trino, and tables created by earlier versions of the connector, which didn't save types. Their columns read back as DOUBLE and VARCHAR.
- A field whose index type isn't the one the connector creates for the saved type, as after an index is dropped and created again outside Trino. It reads back as DOUBLE or VARCHAR.
- Any column, if the connector can't read the key: without read access to it in an ACL, or after Redis evicts it under an `allkeys- eviction` policy. **Creating a table needs write access to the key, such as `~__trino:` in an ACL rule.**

Redis returns a NUMERIC field loaded by name formatted as a double, rounded to 12 significant digits unless it's an integer. DOUBLE and DECIMAL scans use **LOAD *** on HASH tables to retain exact stored values. BIGINT and TIMESTAMP(3) scans load named fields and recover possibly rounded large values with exact HGETs. JSON scans use **DIALECT 3** to read numbers as stored. NUMERIC fields still hold doubles for queries and aggregations in Redis: see [Pushdown](#).

A value its column's type can't hold, such as 4.5 written by another client to a BIGINT column, fails the query with **Value '4.5' is not a valid bigint**. To read the table with the default types instead, delete the key (**DEL `__trino:columns:<table>`**); the first query after `redisearch.table-cache-refresh` seconds has the connector describe the table again, and the queries after it use the default types.

TAG fields created by the connector aren't full-text TEXT fields. They're **CASESENSITIVE** and split values into tags on the ASCII unit separator (`\x1f`) instead of `,`, so Redis can prefilter `=` and **IN** on values containing commas, and doesn't return rows that only differ in case.

Execution and correctness

From SQL to Redis

For a read, the coordinator resolves the index schema and saved Trino column types, then chooses operations that can be evaluated in Redis. Each worker source verifies that its index has finished background indexing and starts an `FT.AGGREGATE` request. Document scans load projected values; grouped queries use `GROUPBY` and `REDUCE`. `FT.CURSOR READ` fetches additional batches until the cursor is exhausted. The connector prefetches one cursor batch while Trino processes the previous batch, and attempts to delete a live cursor when a source closes.

Large eligible document scans can create several sources through [parallel scan planning](#). Pushed-down limits, ordering and aggregations remain single-source operations. Trino applies remaining filters, joins, sorting and expressions to the returned rows. A query that cannot reduce rows in Redis can read every document in the index, even if its final result is small.

Pushdown overview

Redis evaluates these parts of a query. Trino evaluates everything else on the rows Redis returns.

Filters

Indexed predicates combined with `AND` can run in Redis under the following rules. An approximate prefilter can reduce the rows sent to Trino while an exact residual filter remains in Trino. Limits and aggregates that would discard rows before that residual filter remain in Trino too.

NUMERIC fields

`=`, `IN`, `<`, `<=`, `>`, `>=` and `BETWEEN` are eligible on numeric columns and millisecond timestamp columns. Redis indexes numbers as doubles. For large BIGINT equality and IN values, including short inclusive ranges produced by Trino's optimizer, the connector queries widened numeric ranges that include the exact IDs and can also include neighboring IDs. Trino then verifies the stored integers exactly. Unsafe large or unbounded BIGINT ranges, unsafe timestamp bounds, nullable domains and DECIMAL precision above 15 retain a Trino filter. Redis compares REAL values from their stored text (for example `1.1`), rather than the widened binary float (`1.100000023841858`).

TAG fields

`=` and `IN` are eligible for VARCHAR, CHAR, BOOLEAN, DATE and UUID columns. The connector compares values in their stored representation: for example `true`, `2024-01-02` or a lowercase UUID. DATE ranges of at most 1,000 days can become a list of date tags.

Redis splits values using the field's separator, trims surrounding tag whitespace, and ignores case unless `CASESENSITIVE` is set. A tag match therefore does not always mean SQL equality. For an eligible HASH string, an additional exact `FILTER` keeps the equal values, such as `FILTER 'exists(@style) && @style == "Wheat"'`. A value containing the separator, leading/trailing whitespace or unsuitable control characters can require a Trino-only filter.

JSON scans use **DIALECT 3** and keep exact string matches in the connector instead of applying that Redis string **FILTER**; their limit remains in Trino. A JSON boolean TAG is indexed as **true/false**, while the connector reads it as **1/0**, so those equality and IN predicates stay in Trino.

TEXT fields

An equality or IN predicate first finds documents containing the value's words, then applies an exact string **FILTER** where supported. A value made only of stop words, or a field on an index with custom **STOPWORDS**, retains a Trino-only filter. This is SQL equality, not an SQL interface to arbitrary Redis full-text query syntax.

Other string predicates

Trino evaluates **IS NULL**, **IS NOT NULL**, predicates combined with **OR ... IS NULL**, and **VARCHAR** inequality or range comparisons. On eligible **HASH** field names, **LIKE '%literal%'** can use an exact Redis **contains** filter when the literal has no additional **%** or **_** wildcard, **ESCAPE** clause or control character. Other **LIKE** patterns and **JSON LIKE** remain in Trino. Redis wildcard matching alone is insufficient for SQL **LIKE** correctness: its word boundaries, case handling and expansion limits differ.

For control characters, unsupported field names and padded **CHAR** comparisons, exact equality can remain in Trino even when Redis supplies a prefilter. Use **EXPLAIN** to check the chosen plan rather than assuming every indexed predicate is fully consumed.

Aggregations

When aggregation pushdown is enabled, eligible **count(*)**, **min**, **max**, **sum** and **avg** run through **FT.AGGREGATE**, **GROUPBY** and **REDUCE**. Numeric reducer inputs are **REAL**, **DOUBLE** and integer types; **min/max** also support **TIMESTAMP(3)** and **TIMESTAMP(3) WITH TIME ZONE**. **DECIMAL** reducers stay in Trino. If one aggregate in the same aggregation cannot be translated, or Trino must filter input rows itself, Trino computes that aggregation.

count(column), **DISTINCT** aggregates and filtered aggregates stay in Trino. Redis's **COUNT** counts documents, whereas SQL **count(column)** excludes **NULL** values. Trino also evaluates **HAVING** after the groups are returned.

For supported **SUM** and **AVG** fields, the connector computes a sum of present values and their count using **case()** and **exists()** expressions. An empty group returns SQL **NULL**; **AVG** divides by the count of values, rather than the number of documents. This avoids Redis reducer behavior on sharded databases where a shard with no values can contaminate a sum or average. The **case()** path requires Redis 8.2 or later, or unstable features enabled on Redis 8.0/Stack 7.4; the connector probes it. Without that capability, or with an unsupported field name, the older native **SUM/AVG** path remains, which is correct for this null-handling case on one shard. Test sharded nullable data before relying on that older path.

Redis computes on doubles and can format floating results and group keys rounded to 12 significant digits. For eligible **DOUBLE/REAL** outputs and keys, the connector also requests integer mantissa/exponent expressions and reconstructs the computed double. This preserves the Redis double result; it does not give arbitrary-precision decimal arithmetic. With **RESP2**, eligible floating aggregations stay in Trino because precision can be lost between shards before the coordinator replies.

NUMERIC-backed BIGINT grouping stays in Trino so adjacent large integers do not collapse into one double group. DECIMAL grouping and floating grouping with unsupported field names also stay in Trino. Other pushed-down BIGINT computations are exact only within double's integer precision range; reading exact stored values does not recover precision already lost in an aggregate. Widened INTEGER/SMALLINT/TINYINT SUM inputs have conservative document-count bounds; an unknown or excessive count retains Trino computation.

GROUP BY pushdown requires a known cached document count no greater than `redisearch.aggregation-group-limit`. That unfiltered count bounds the possible number of groups, even if WHERE is selective. Keep the configured budget within the server's `MAX_AGGREGATE_GROUPS` limit. Index growth after caching can still cause a server error; the query fails rather than replaying partially delivered output. Global aggregates have no GROUP BY cardinality budget.

Arithmetic aggregates

Eligible HASH expressions such as `sum(quantity * extendedprice)` and `avg(quantity * extendedprice)` use a Redis `APPLY` step before reducing. Supported operations are `+`, `-`, `*`, `/` and unary minus on DOUBLE NUMERIC inputs, integer NUMERIC inputs cast to DOUBLE, and DOUBLE constants. This path needs `case()` so that a missing input yields SQL NULL behavior.

Expressions on integer types that must retain SQL overflow behavior, REAL arithmetic, and `min/max` of arithmetic stay in Trino. The latter also avoids differences in ordering NaN values. A document scan returning an eligible arithmetic expression computes it in the connector from the stored values.

Limits and ordering

`LIMIT` can run in Redis when an exact residual filter is not required in Trino. Without an `ORDER BY`, SQL does not guarantee which matching rows a limit returns.

For HASH indexes, `ORDER BY ... LIMIT` uses Redis `SORTBY` when all sort keys are indexed NUMERIC columns of type DOUBLE, REAL, INTEGER, SMALLINT or TINYINT, with supported field names and `NULLS LAST`. Trino sorts the returned candidates to produce the final order. BIGINT and DECIMAL sort keys do not use this path because Redis double ordering can lose precision. JSON ordering stays in Trino.

A separate HASH optimization supports one `TIMESTAMP(3)` sort key, `NULLS LAST`, and a limit of at most 1,000. It scans keys and exact order values, keeps a bounded set of candidates, and loads their projected fields with pipelined `HMGET` only after selection finishes. This reduces wide-row retrieval but still scans the matching keys. `TIMESTAMP WITH TIME ZONE`, multiple timestamp sort keys, larger limits and other null ordering remain in Trino. Selection checks the configured timeout over the scan and emits no candidates if it fails.

Joins and optimizer statistics

Trino executes joins, including joins between Redis and another catalog. For an eligible document scan, dynamic filtering adds join keys collected from the other side to the Redis query, using the same indexed-field rules as static filters. The scan can wait up to `redisearch.dynamic-filtering.wait-timeout` (20 seconds by default). Large dynamic domains are compacted with a

threshold of 256, so a range prefilter can replace many individual values; Trino still evaluates the join. A pushed-down limit or aggregation does not use this document-scan dynamic filtering path.

Table row-count statistics come from cached `FT.INFO num_docs` values. They help Trino choose join order and distribution, but do not estimate predicate selectivity: the filtered estimate can still be the whole index count. A missing or stale statistic affects planning; it does not intentionally cap the returned rows.

Timeouts, incomplete results and retries

`redisearch.query-timeout-ms` sets the per-query Redis aggregation timeout in milliseconds; `0` uses the server default. For example, `redisearch.query-timeout-ms=30000` requests 30 seconds. Trino query limits, Redis server policy and client connection/command timeouts remain separate constraints; a larger aggregation timeout does not remove them. Where the deployment permits it, configure Redis's timeout policy to fail instead of returning partial results, following [Redis Query Engine configuration](#). The connector rejects aggregation replies containing server warnings with `RESEARCH_INCOMPLETE_RESULT`.

A lost cursor-read reply fails with `RESEARCH_CURSOR_REPLY_LOST` rather than replaying the read: Redis may already have consumed its batch. Restore server/network health and rerun the complete query. The connector can reconnect for later queries and performs bounded retries for idempotent cursor deletion, but does not resume a failed result stream. A client that has received rows before a later scan error must treat the query as failed and discard those rows.

Read consistency

Redis cursors and the connector's follow-up exact hash reads do not provide snapshot isolation. Concurrent updates or deletes can change the values observed during a scan. Parallel scans additionally require a stable partition field to avoid moving documents between ranges while they are read. For repeatable analytical checks, use a quiescent dataset or an isolated copy and compare complete query results.

Performance and concurrency

Start with the execution plan

Start with the defaults: aggregation pushdown enabled, automatic splits, eight scan connections per node and 1,000 rows per cursor batch. A selective query or supported aggregation usually benefits most from returning fewer rows from Redis. A full scan can instead be limited by network transfer, exact-value reads or conversion into Trino pages. Use `EXPLAIN` to see the chosen filters, limits, sort keys and aggregates, then `EXPLAIN ANALYZE` on a representative read to measure its execution. `EXPLAIN ANALYZE` executes the query.

Connector source metrics include `redis.aggregate.requests`, `redis.cursor.requests`, `redis.rows.received`, `redis.exact-hash-reads`, `redis.request-wall-time` and `redis.row-conversion-time`. Inspect these alongside Redis CPU, query latency and Trino worker activity. Request wall time includes waiting and network time; it is not a measurement of Redis CPU time.

For unbounded scans of at most two requested columns, `redisearch.narrow-scan-cursor-count` can reduce cursor round trips. Larger batches retain more rows in memory; test them with concurrent queries before adopting them. Selecting only needed columns also reduces work, although exact DOUBLE and DECIMAL HASH reads can require `LOAD *`.

Sharing Redis with application traffic

A pushed-down global aggregate must process all matching documents before returning its result. `WITHCURSOR COUNT` batches the output, not the global computation. More client connections or smaller batches do not by themselves remove server-side contention. Evaluate [Redis query performance factor](#) independently of connector scan splits. Additional query workers consume CPU and can increase overhead; measure the target workload.

When large aggregates interfere with latency-sensitive traffic, compare this catalog setting:

```
redisearch.aggregation-pushdown.enabled=false
```

Trino then computes aggregates from document batches while supported filters still run in Redis. This transfers more rows and can make analytical queries slower. Use Trino resource groups to bound analytical concurrency, and measure application tail latency as well as SQL throughput.

Benchmark evidence

The repository retains timings and reproduction instructions rather than promising a workload-independent speedup. The README distinguishes historical CI timings, pre-v0.4.0 comparisons and newer full-data experiments. Treat each result as evidence for its pinned build, dataset, server configuration and concurrency.

Report	Use it to examine
Aggregation contention	Point-query latency while analytical aggregations run, and query-worker tradeoffs.
Parallel scans	Partitioned scan throughput and single-source comparisons.
QPF, splits and concurrency	A controlled 5-million-row Redis Software run across query-worker, split and client-count settings.
ClickBench results	Query coverage, correctness, full-data timings and retained evidence.

Parallel document scans

The connector uses Lettuce for Redis communication. Lettuce supports multiple CPU cores through Netty I/O event loops and computation thread pools. Its default pool sizes follow the available CPU count, with a minimum of two threads, unless configured otherwise. See [Lettuce client resources](#) for the threading settings. A physical connection processes its network events on one event loop; additional client threads do not automatically partition a query or create more Trino sources.

Layer	Work it parallelizes
Lettuce	Redis network I/O and protocol decoding across connections.
Trino connector	Independent scan splits, each with a cursor, processed by separate Trino drivers/workers.
Redis Query Engine	Server-side query computation through query workers.

The connector's existing connection pool supports concurrency between scans. Multiple splits let a single document scan use several connections and Trino source pipelines. Tune Redis query workers independently and measure the combined configuration; client-side parallelism alone does not determine server-side query concurrency.

Automatic split selection is the default (`redisearch.scan-splits=0`). The planner targets at least 250,000 matching rows per split and selects the largest count allowed by the estimated CPU capacity (active Trino workers times coordinator CPU count), the connection pool (eight by default), and a hard ceiling of 64. Small tables and selective filters use one split. A filter's match count is checked before choosing parallelism. For HASH indexes without explicit boundaries, the planner examines up to three indexed NUMERIC fields, obtains their MIN/MAX in one bounded statistics query, and checks the counts of the proposed ranges. A field is rejected if one partition would contain at least 80% of the matching documents; the planner then tries another candidate.

Discovery results, including unsupported cases, are cached for five minutes (up to 256 entries). Statistics queries have a ten-second server timeout and count probes have a one-second timeout; unavailable or partial statistics fall back to one split. The first scan can therefore pay a discovery cost; subsequent scans can reuse discovery results. Selective-filter count probes still run before the discovery cache is consulted, so a cached plan does not eliminate all planning requests.

MIN/MAX discovery is a server-side aggregate and can delay concurrent point traffic when Redis query workers are disabled; configured boundaries avoid this discovery work. This is a cost heuristic, not a guarantee of the optimal count under every load. Use the benchmark and explicit

overrides when tuning a shared deployment. JSON arrays support explicit partition fields and cut points; automatic MIN/MAX discovery currently applies to HASH indexes.

Set `redisearch.scan-splits=1` to force one source. To override automatic selection and supply known boundaries:

```
redisearch.scan-splits=4
redisearch.scan-connections=4
redisearch.scan-partition-field=ordinal
redisearch.scan-partition-boundaries=1250000,2500000,3750000
```

This partitions one document scan into four Trino sources with independent cursors and distinct scan connection slots on each worker. The NUMERIC field must already be indexed; the connector does not alter the index. Explicit boundaries skip discovery. Choose cut points from the field's distribution; uneven partitions still return complete results but can limit the speedup. If fewer splits are allowed, the connector selects evenly spaced cut points from the configured list and merges the other intervals.

The lowest partition includes values below the first cut and documents without an indexed value; the highest includes every value above the last cut. For multi-valued JSON, partitions use differences of nested range matches, so documents matching multiple ranges appear once. All partitions preserve static and dynamic filters. Each source rechecks its index schema before querying and fails if the partition field has become unsafe.

Pushed-down LIMIT, ORDER BY/top-N, grouping and reducers always use one split. Trino can still compute joins, unsupported expressions and exact DISTINCT from the parallel document sources. The normal cursor timeout, failure, exact BIGINT recovery and best-effort deletion behavior applies to each source.

The pool bounds connections per worker; the split bound applies per query, not to the number of simultaneous queries. Configure Trino resource groups to bound overall analytical concurrency when sharing Redis with latency-sensitive traffic. Parallel scans do not provide snapshot isolation: keep the partition field stable during a scan, as moving a document across cut points during execution can cause omissions or repetitions.

Limitations

The statement and pushdown references above describe the supported paths. Additional limits to keep in mind:

- Redis Query Engine indexes define the table set; arbitrary keyspace structures are not SQL tables.
- JSON writes are limited to document deletion; INSERT and UPDATE require HASH indexes.
- GEO, GEOSHAPE and VECTOR fields read as VARCHAR. This connector does not translate SQL into native geospatial or vector-similarity queries.
- Type discovery samples unindexed fields and does not infer an exhaustive schema from every document.
- NUMERIC indexing and Redis aggregation use doubles; stored-value recovery does not make every pushed-down numeric computation exact.
- Scans are not snapshots, and writes do not provide multi-row atomicity or automatic rollback of completed changes.
- Flex limitations are deployment-specific and require execution-level validation before support can be claimed.

Queries known to return wrong results or fail are tracked as [open issues labeled bug](#).

Clients

JDBC driver

The Trino JDBC driver allows users to access Trino from Java-based applications, and other non-Java applications running in a JVM.

Refer to the [Trino documentation](#) for setup instructions.

The following is an example of a JDBC URL used to create a connection to Redis SQL Trino:

```
jdbc:trino://localhost:8080/redisearch/default
```

The connection targets Trino, which then connects to Redis. The path selects the catalog and schema; change `redisearch` if you used another catalog filename. Use a JDBC driver matching the Trino server version for this deployment. For a secured Trino endpoint, configure the driver's TLS and authentication options separately from the Redis catalog's credentials.

Tableau and other visualization clients

Configure a supported Trino/Presto connection to the Trino coordinator host and port, then choose the Redis catalog and its configured schema. Redis's database port is not a JDBC endpoint. Refer to the [Tableau connection instructions](#) for the driver's installation and connection fields. For Apache Superset or another SQL client, use its Trino integration and the same catalog/schema. Validate an ordinary SELECT in the Trino CLI first so that driver setup can be diagnosed separately from connector or Redis errors.

Trino CLI

Use the CLI bundled with the stock container, or download the executable CLI JAR for Trino 483. For a locally installed CLI:

```
trino --server http://localhost:8080 --catalog redisearch --schema default
```

```
SHOW CATALOGS;  
SHOW SCHEMAS FROM redisearch;  
SHOW TABLES FROM redisearch.default;  
DESCRIBE redisearch.default.guide_beers;  
EXPLAIN SELECT name, abv FROM redisearch.default.guide_beers WHERE abv > 4;
```

The last two statements require the [example index](#) or an equivalent table of your own. See [the Trino CLI reference](#) for TLS, authentication and output options.

Troubleshooting

Start by identifying the failing layer: Trino startup, catalog discovery, query execution or the external client. Check Trino's startup log, query failure message and Redis endpoint reachability from the affected node. Test Redis commands with the same endpoint, TLS mode and service account used by Trino. Keep credentials out of diagnostic logs and shared reports.

Symptom	What to check
Catalog fails to start	Confirm Trino 483, Java 25, all plugin dependency JARs and valid properties. Remove defunct settings. Check that each node sees the same catalog and certificates.
Unknown command <code>FT._LIST</code> or <code>FT.AGGREGATE</code>	Confirm the database has a compatible Query Engine. Plain Redis without Search and the documented Flex variants cannot satisfy these paths.
<code>SHOW TABLES</code> is empty	Check <code>FT._LIST</code> using the service account. Keys without an index do not create SQL tables. Check the catalog, schema and configured case matching.
Table exists but expected rows are absent	Inspect its key prefix, HASH/JSON key type and <code>FT.INFO num_docs</code> . <code>CREATE TABLE</code> skips the initial scan of existing keys. <code>INSERT</code> does not derive keys from ID columns.
<code>REINDEXING_INDEX_NOT_READY</code>	Wait for <code>FT.INFO indexing=0</code> ; inspect indexing failures and percent indexed if it does not finish. Do not disable the guard to accept incomplete results.
New columns do not appear	Check the index schema and saved type key, then allow the background metadata refresh to complete. The default refresh interval is 60 seconds; <code>0</code> disables caching.
<code>NOPerm</code> , authentication or TLS failure	Check both command/key permissions and credentials. Use <code>rediss://</code> for TLS; mount trusted CA/client certificate files at the configured paths on each node.
Cluster connection fails	Enable <code>redisearch.cluster</code> only for an OSS Cluster API endpoint, and verify every advertised node address is reachable from Trino.
Timeout or <code>REINDEXING_INCOMPLETE_RESULT</code>	Inspect server warnings, load and timeout policy. Choose an appropriate query timeout; reject partial results and rerun the whole query after addressing the cause.
<code>REINDEXING_SHARD_TOPOLOGY_UNAVAILABLE</code>	Check Redis coordinator and shard health; compare a fresh direct connection to the same database. Retry the complete query after topology health is restored.
<code>REINDEXING_CURSOR_REPLY_LOST</code>	A cursor read may have consumed rows before its reply was lost. Restore connection health, discard the failed result and rerun the whole query rather than replaying a batch.

Symptom	What to check
GROUP BY fails at the server limit	Keep <code>redisearch.aggregation-group-limit</code> within the server's group budget. Cache staleness or table growth can outgrow a previously valid plan. Disable aggregation pushdown to compare a Trino aggregation plan.
Unexpected type or invalid value	Check <code>__trino:columns:<index></code> and values written by other clients. A saved BIGINT column containing <code>4.5</code> is an error, not a silently truncated integer.
SQL is slow but returns few rows	Use <code>EXPLAIN</code> to check for residual filters, unsupported aggregates or sorts. A small final output can still require a full scan. Compare source metrics and Redis/Trino CPU before changing batch or split settings.

For a report, include connector and Trino versions, Redis/Search versions, HASH versus JSON, connection mode, a redacted catalog, the failing SQL and its plan/error, and a small reproducible dataset. Include concurrency and timeout settings for performance or intermittent failures. File the report in [the project issue tracker](#).

Building from source

Run these commands to build the Trino connector for Redisearch from source (requires Eclipse Temurin or Oracle JDK 25):

```
git clone https://github.com/redis-field-engineering/redis-sql-trino.git
cd redis-sql-trino
./mvnw clean package -DskipTests
```

The distribution is `target/redis-sql-trino-0.4.2.zip`; the unpacked plugin directory is `target/redis-sql-trino-0.4.2/`. The build above skips tests. Run `./mvnw test` with Docker available for the integration suite; the default test environment provisions Redis Software. The README's [build and benchmark instructions](#) describe profiles for alternate Redis targets and the query benchmark. A source build uses the version in `pom.xml`, which can differ from the release version printed in this published guide. Set `REDIS_SQL_TRINO_VERSION` to the actual built version before using the repository's Compose file, rather than relying on its default directory name.

Generate this guide's HTML and PDF without running connector tests:

```
./mvnw process-resources
```

Outputs are written to `target/generated-docs/`. The repository's Publish workflow builds this same source with a release version and deploys that directory to GitHub Pages.

Single-node Linux walkthrough

This walkthrough installs a single-node Trino server from the release archives on Linux. It assumes an existing Query Engine database. Create the installation and data directories with administrator help if needed, then give the Trino service user read access to the installation and write access to the data directory. For a local trial with fewer host dependencies, use the [Docker quick start](#).

Install Java

Trino requires a 64-bit version of Java 25. Use [Eclipse Temurin](#) Java 25, at least 25.0.1, consistent with Trino and the project build. Confirm that `JAVA_HOME` and `java` point to that JDK; Java 26 is not a supported substitute. See [Trino deployment requirements](#).

```
java -version
```

Set up Trino

Download the Trino server tarball and unpack it.

```
wget https://repo1.maven.org/maven2/io/trino/trino-server/483/trino-server-483.tar.gz
mkdir -p /usr/lib/trino
tar xzvf trino-server-483.tar.gz --directory /usr/lib/trino --strip-components 1
```

Trino needs a data directory for storing logs, etc. It is recommended to create a data directory outside of the installation directory, which allows it to be easily preserved when upgrading Trino.

Create a data directory

```
mkdir -p /var/trino
```

Create an `etc` directory inside the installation directory to hold configuration files.

```
mkdir -p /usr/lib/trino/etc
```

Create a `node.properties` file.

/usr/lib/trino/etc/node.properties

```
node.environment=demo
node.id=redis-trino-demo-1
node.data-dir=/var/trino
```

Create a JVM config file. The following uses a 4 GiB heap for a small trial; the host needs additional memory for native allocations, the operating system and other services. Size production heaps

using Trino's deployment guidance.

/usr/lib/trino/etc/jvm.config

```
-server
-Xmx4G
-XX:G1HeapRegionSize=32M
-XX:+ExplicitGCInvokesConcurrent
-XX:+ExitOnOutOfMemoryError
-XX:+HeapDumpOnOutOfMemoryError
-XX:-OmitStackTraceInFastThrow
-XX:ReservedCodeCacheSize=512M
-XX:PerMethodRecompilationCutoff=10000
-XX:PerBytecodeRecompilationCutoff=10000
-Djdk.attach.allowAttachSelf=true
-Djdk.nio.maxCachedBufferSize=2000000
--add-modules=jdk.incubator.vector
```

Create a config properties file. This node acts as both coordinator and worker. The example HTTP endpoint has no authentication; use it only on a trusted local network and configure [Trino security](#) before exposing it to other users.

/usr/lib/trino/etc/config.properties

```
coordinator=true
node-scheduler.include-coordinator=true
http-server.http.port=8080
discovery.uri=http://localhost:8080
```

Create a logging configuration file.

/usr/lib/trino/etc/log.properties

```
io.trino=INFO
```

Set up Redis SQL Trino

Download latest [release](#) and unzip without any directory structure under `plugin/redisearch`:

```
wget https://github.com/redis-field-engineering/redis-sql-trino/releases/download/v0.4.2/redis-sql-trino-0.4.2.zip
unzip -j redis-sql-trino-0.4.2.zip -d /usr/lib/trino/plugin/redisearch
```

Create a etc/catalog subdirectory:

```
mkdir -p /usr/lib/trino/etc/catalog
```

Create a Redisearch connector configuration file:

```
/usr/lib/trino/etc/catalog/redisearch.properties
```

```
connector.name=redisearch  
redisearch.uri=redis://localhost:6379
```

Change and/or add [properties](#) as needed.

Start Trino

Start the Trino server:

```
/usr/lib/trino/bin/launcher run
```

Run Trino CLI

Download and run the CLI as the current user:

```
wget https://repo1.maven.org/maven2/io/trino/trino-cli/483/trino-cli-483-  
executable.jar -O ./trino  
chmod +x ./trino  
./trino --catalog redisearch --schema default
```

List tables and run a SQL query:

```
SHOW TABLES;  
-- Replace guide_beers with an index present in your database.  
SELECT __key, * FROM guide_beers LIMIT 10;
```